

Apache Paimon paimon-core 架构设计文档

核心架构设计 · 设计模式 · 流程图 · 时序图 · 调用链

Module: paimon-core (release-1.3.1)

Generator: AI Architecture Analyzer

Date: 2026-03-26

Chapter 1 Module Overview

1.1 Module Positioning

paimon-core is the core storage engine module of Apache Paimon, a data lake storage system designed for high-throughput, low-latency streaming and batch data processing. This module provides the complete data lifecycle management including table creation, schema evolution, data writing, compaction, snapshot management, and data reading. Built on an LSM (Log-Structured Merge) Tree architecture, it supports both append-only and primary-key table models.

The module contains approximately 789 Java source files organized into 36+ top-level packages, making it the largest and most critical module in the Apache Paimon ecosystem. It sits between the public API layer (paimon-api) and the compute engine connectors (Flink/Spark), serving as the fundamental storage engine that all higher-level integrations depend on.

1.2 Core Objectives

- High-throughput streaming writes — Support real-time data ingestion with LSM tree-based write path, enabling millions of records per second per partition
- ACID transaction support — Provide snapshot-based MVCC with optimistic concurrency control for consistent reads and atomic commits
- Flexible merge strategies — Support multiple merge functions (deduplicate, partial-update, aggregate, first-row, lookup) for diverse use cases
- Automatic compaction — Implement universal compaction and clustering strategies to maintain read performance while minimizing write amplification
- Schema evolution — Support adding/dropping/renaming columns, type widening, and nested type evolution without data rewriting
- Multi-engine compatibility — Provide a clean FileStore abstraction consumable by Flink, Spark, Hive, Trino, and other compute engines

1.3 Package Structure Overview

Package	Files	Description
mergetree/	114	LSM Merge Tree engine: writer, reader, compaction, merge functions
table/	85	Table abstractions: FileStoreTable, source, sink, system tables
operation/	60	Core operations: scan, write, commit, compact, expire
io/	55	Data file I/O: readers, writers, file formats, key-value encoding

Package	Files	Description
catalog/	43	Catalog implementations: FileSystem, JDBC, REST, Caching
manifest/	30	Manifest system: ManifestFile, ManifestList, ManifestEntry tracking
schema/	28	Schema management: creation, evolution, validation
iceberg/	29	Iceberg format compatibility layer
stats/	25	Statistics collection and management
tag/	20	Tag (named snapshot) management and auto-creation
deletionvectors/	18	Deletion vector support for row-level deletes
compact/	15	Compaction strategies and scheduling
utils/	45	Utility classes: snapshot management, path helpers, etc.
privilege/	12	Privilege/permission management
consumer/	8	Consumer ID tracking for streaming reads
index/	20	Bucket index and hash index implementations

Chapter 2 Core Architecture Design

2.1 Layered Architecture

The paimon-core module follows a strict four-layer architecture: API Layer, Table Layer, Operation Layer, and Storage Layer. Each layer has well-defined interfaces with clear responsibilities, enabling independent evolution and testing.

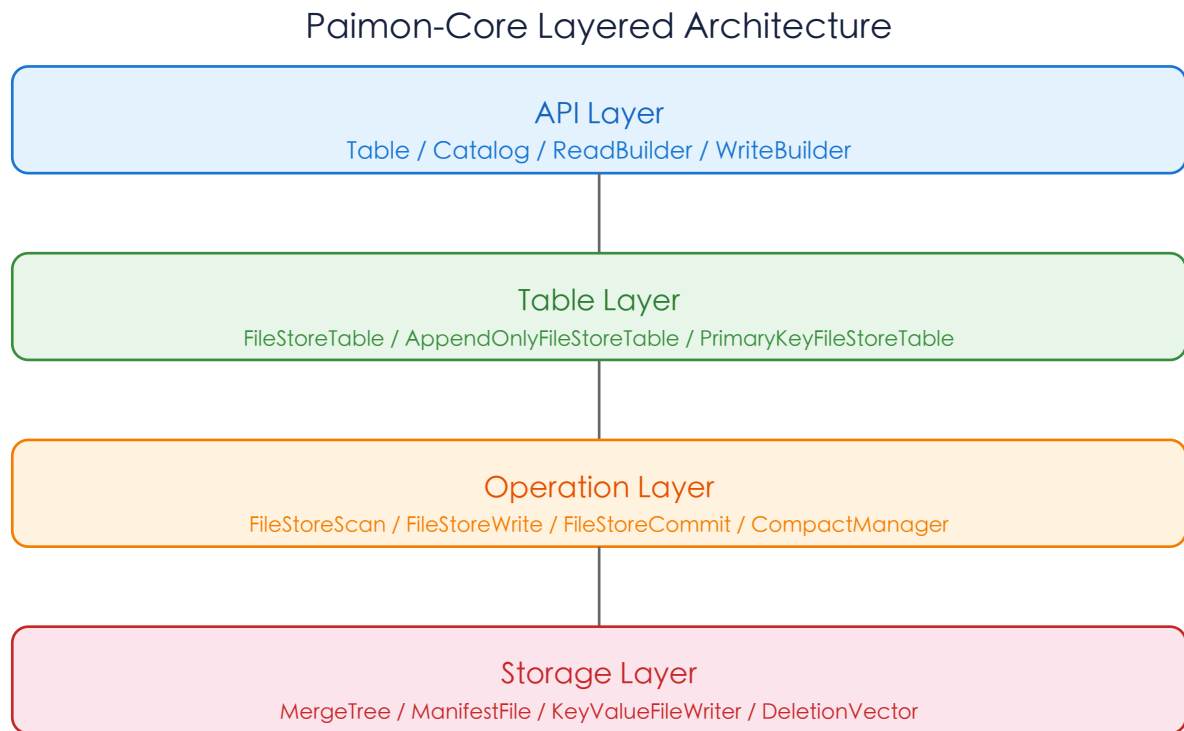


Fig 2-1: Paimon-Core Four-Layer Architecture

2.2 Core Interface Hierarchy: Table

The Table interface is the primary entry point for users and compute engines. It follows a deep inheritance hierarchy that progressively adds storage-specific capabilities:

```
Table (paimon-api, @Public)
+-- InnerTable
+-- DataTable
+-- FileStoreTable
+-- AbstractFileStoreTable
+-- AppendOnlyFileStoreTable
+-- PrimaryKeyFileStoreTable
```

Table defines public read/write builder APIs (`newReadBuilder`, `newBatchWriteBuilder`, `newStreamWriteBuilder`). `FileStoreTable` extends this with `FileStore` access, `ConsumerManager`, `TagAutoManager`, and schema management. `AbstractFileStoreTable`

provides the template method implementation, while the two concrete subclasses specialize for append-only vs primary-key semantics.

2.3 Core Interface: FileStore

FileStore is the central storage engine interface that encapsulates all storage operations. It is parameterized by the record type T (InternalRow for append-only, KeyValue for primary-key).

```
public interface FileStore<T> {  
    FileStorePathFactory pathFactory();  
    SnapshotManager snapshotManager();  
    ManifestFile.Factory manifestFileFactory();  
    ManifestList.Factory manifestListFactory();  
    PartitionExpire newPartitionExpire(String commitUser);  
    FileStoreScan newScan();  
    FileStoreWrite<T> newWrite(String commitUser);  
    FileStoreCommit newCommit(String commitUser);  
    TagDeletion newTagDeletion();  
}
```

Two concrete implementations exist: AppendOnlyFileStore for insert-only workloads, and KeyValueFileStore for upsert workloads with merge semantics. The AbstractFileStore base class provides common infrastructure including path management, manifest handling, and snapshot operations.

2.4 Catalog System

The Catalog interface provides metadata CRUD operations for databases and tables. The hierarchy employs the Decorator pattern extensively:

```
Catalog (interface, 1777 lines)  
+-- AbstractCatalog (common CRUD logic)  
+-- FileSystemCatalog (file-based metadata)  
+-- JdbcCatalog (JDBC-based metadata)  
+-- RESTCatalog (HTTP REST API)  
+-- DelegateCatalog (Decorator base)  
+-- CachingCatalog (metadata caching)  
+-- PrivilegedCatalog (permission checks)
```

2.5 Operation Layer Architecture

The Operation Layer provides three core operation interfaces that form the read-write-commit triangle:

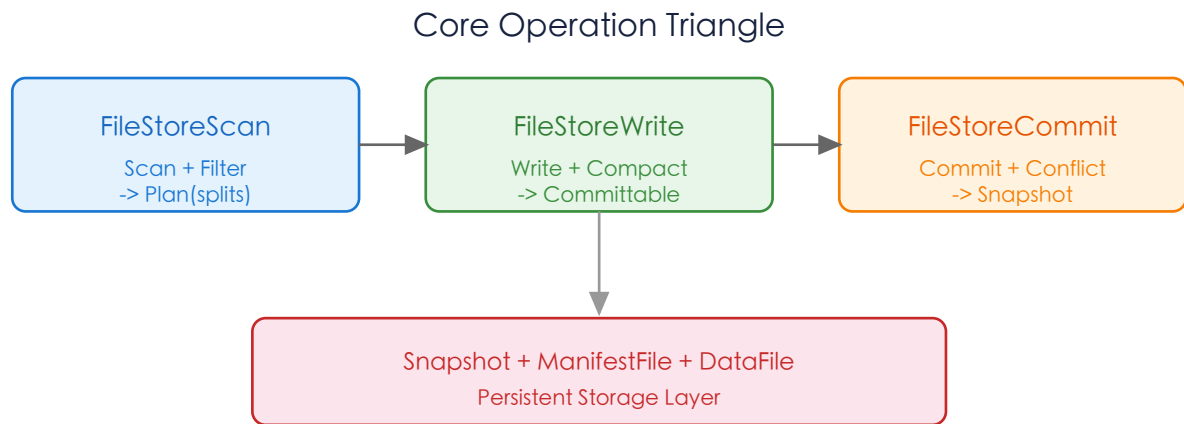


Fig 2-2: Core Operation Triangle

FileStoreScan reads manifest files to produce a Plan containing DataFileMeta splits. FileStoreWrite accepts records, buffers them in a MemTable, flushes to sorted runs, and triggers compaction. FileStoreCommit atomically commits the resulting file changes to a new snapshot with optimistic concurrency control and conflict detection.

Chapter 3 Design Patterns Analysis

3.1 Template Method Pattern — AbstractFileStore / AbstractFileStoreTable

The Template Method pattern is the most pervasive pattern in paimon-core. AbstractFileStore defines the skeleton of storage operations while deferring type-specific details to subclasses.

Pattern Manifestation:

- AbstractFileStore provides common implementations for pathFactory(), snapshotManager(), manifestFileFactory(), while leaving newScan(), newWrite() abstract for AppendOnlyFileStore and KeyValueFileStore
- AbstractFileStoreTable implements the common table logic (schema validation, option parsing, copy operations) while subclasses define the specific FileStore creation and read/write builder construction
- AbstractCatalog defines the workflow for createTable/dropTable with validation steps, delegating actual metadata persistence to FileSystemCatalog/JdbcCatalog

3.2 Strategy Pattern — MergeFunction / CompactStrategy

The Strategy pattern allows runtime selection of merge and compaction behaviors:

Strategy Interface	Implementations	Use Case
MergeFunction	DeduplicateMergeFunction	Keep the latest record by key (default)
MergeFunction	PartialUpdateMergeFunction	Merge non-null fields from multiple records
MergeFunction	AggregateMergeFunction	Apply aggregation functions (sum, max, etc.)
MergeFunction	FirstRowMergeFunction	Keep the first occurrence of each key
MergeFunction	LookupMergeFunction	Lookup-based merge with external state
CompactStrategy	UniversalCompaction	Size-ratio based universal compaction
CompactStrategy	ForceUpLevel0Compaction	Force compact level-0 files

3.3 Decorator Pattern — DelegateCatalog / CachingCatalog

The Decorator pattern is used extensively in the Catalog system to add cross-cutting concerns without modifying existing implementations:

- DelegateCatalog wraps any Catalog implementation, forwarding all calls to the delegate

- CachingCatalog extends DelegateCatalog to add metadata caching for table/database lookups
- PrivilegedCatalog extends DelegateCatalog to add permission checks before operations

This enables flexible composition: a FileSystemCatalog can be wrapped with CachingCatalog and then PrivilegedCatalog, adding both caching and access control transparently.

3.4 Factory Pattern — FileStoreTableFactory / MergeFunctionFactory

Multiple Factory patterns are used to create complex objects with proper initialization:

- FileStoreTableFactory creates the appropriate FileStoreTable based on table type (primary-key vs append-only)
- MergeFunction.Factory (inner interface) creates MergeFunction instances with proper configuration for each merge type
- ManifestFile.Factory / ManifestList.Factory create manifest I/O objects with correct file format and compression settings
- KeyValueFileWriterFactory creates file writers with appropriate format (ORC/Parquet/Avro) and encoding settings

3.5 Builder Pattern — ReadBuilder / WriteBuilder / ScanBuilder

The Builder pattern provides fluent APIs for constructing complex read/write operations:

```
table.newReadBuilder()
  .withFilter(predicate)
  .withProjection(projection)
  .withBucketFilter(bucketFilter)
  .newRead() // -> TableRead
  .createReader(split) // -> RecordReader
```

3.6 Observer Pattern — CommitCallback / IcebergCommitCallback

The commit process uses callbacks to notify interested parties of successful commits. IcebergCommitCallback maintains Iceberg-format metadata in parallel with Paimon metadata, enabling dual-format compatibility without coupling the core commit logic to Iceberg specifics.

Chapter 4 Core Flow Charts

4.1 Data Write Flow

The data write flow spans from user record ingestion through the LSM tree write buffer to persistent data files. This is the critical hot path for streaming data ingestion.

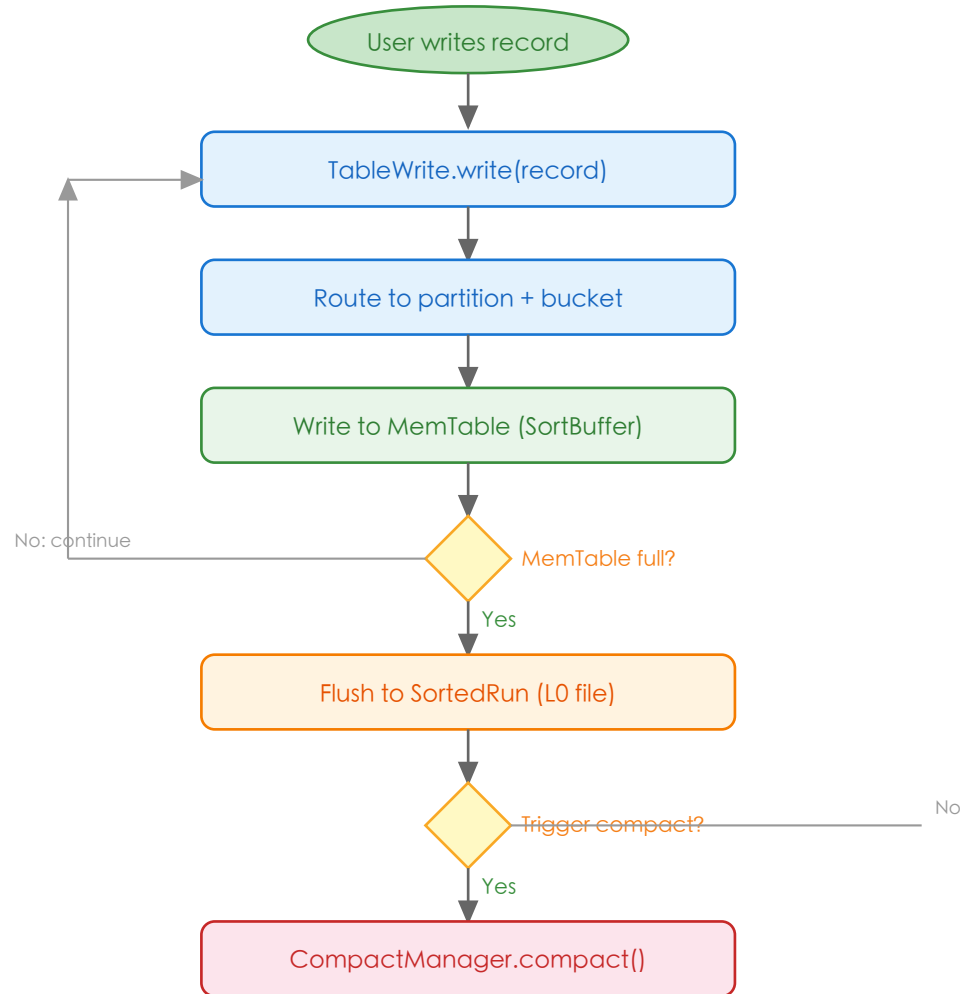


Fig 4-1: Data Write Flow

4.2 Commit Flow

The commit flow converts pending file changes into an atomic snapshot. It implements optimistic concurrency control with manifest-based conflict detection.

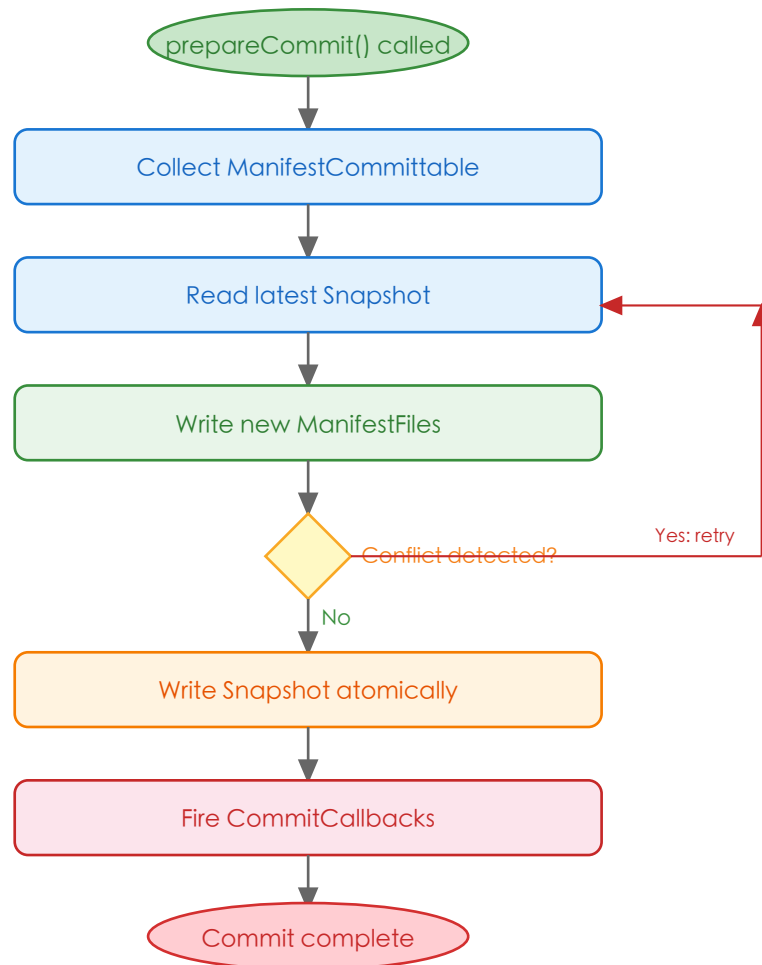


Fig 4-2: Commit Flow with Optimistic Concurrency Control

4.3 Compaction Flow

Compaction is triggered when the number of sorted runs at level 0 exceeds the threshold. The universal compaction strategy selects files based on size ratios.

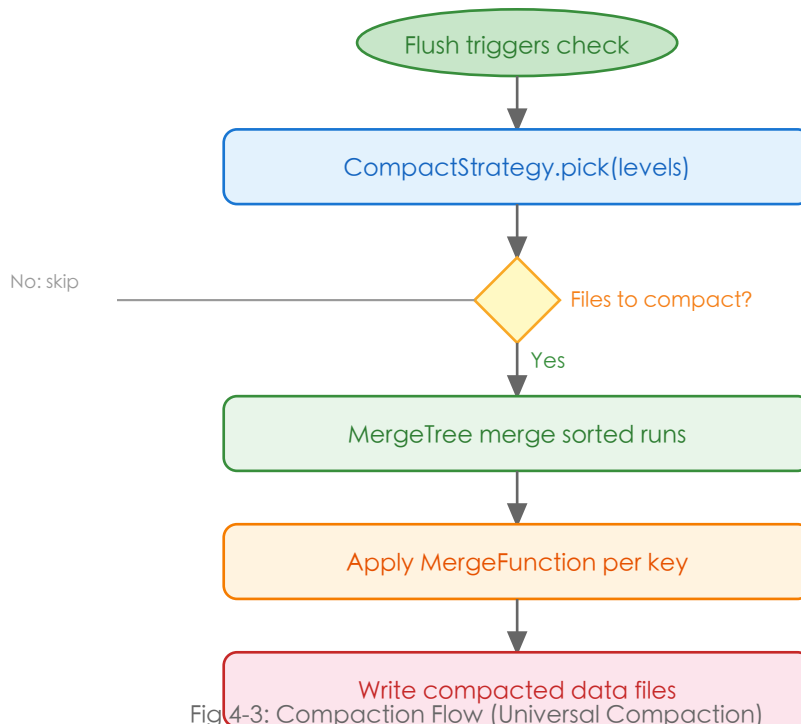


Fig 4-3: Compaction Flow (Universal Compaction)

Chapter 5 Sequence Diagrams

5.1 Write Path Sequence Diagram

This sequence diagram shows the complete write path from the user API call through to persistent file creation. The key participants are the Table, FileStoreWrite, MergeTreeWriter, MemTable, and FileWriter.

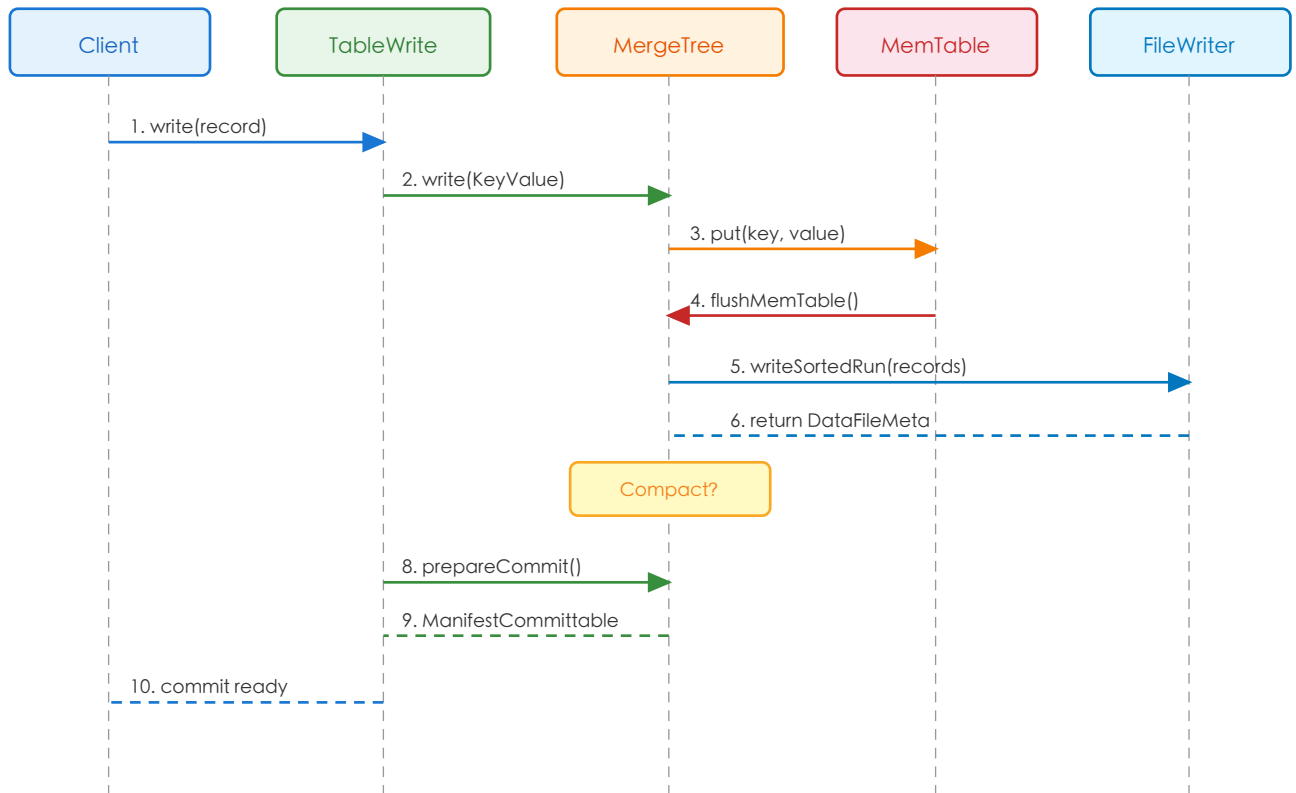


Fig 5-1: Write Path Sequence Diagram

5.2 Scan/Read Path Sequence Diagram

The read path starts with a scan plan generation and then reads data from the planned splits. It involves manifest file reading, predicate pushdown, and merge-tree-based record assembly.

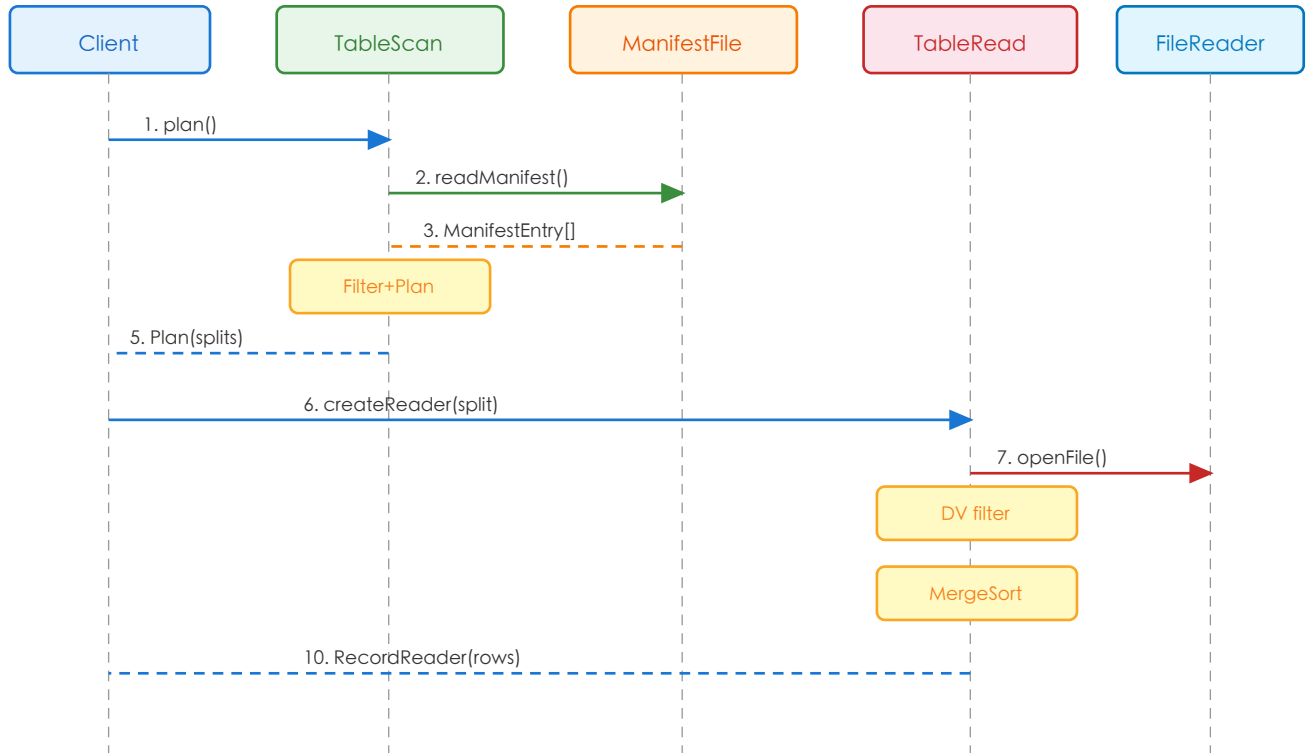


Fig 5-2: Scan/Read Path Sequence Diagram

Chapter 6 Key Scenario Call Chains

6.1 Scenario 1: Table Creation

Trigger: User executes CREATE TABLE via SQL engine (Flink/Spark) or Catalog API

Call Chain:

Step	Class.Method	Description
1	Catalog.createTable(identifier, schema)	Entry point: validate database exists, check table not exists
2	AbstractCatalog.createTableImpl()	Template method: delegate to concrete catalog
3	FileSystemCatalog.createTableImpl()	Create table directory in filesystem
4	SchemaManager.createTable(schema)	Validate schema, assign field IDs, resolve options
5	SchemaManager.commit(schema)	Atomically write schema-0 file to table/schema/directory
6	SchemaValidation.validateTableSchema()	Validate primary keys, partition keys, bucket configuration

6.2 Scenario 2: Streaming Data Write

Trigger: Flink streaming job continuously writes records to a Paimon table

Call Chain:

Step	Class.Method	Description
1	TableWrite.write(InternalRow)	Accept a record, extract partition and bucket key
2	AbstractFileStoreWrite.write()	Route record to correct partition-bucket writer
3	MergeTreeWriter.write(KeyValue)	Convert row to KeyValue, put into SortBuffer(MemTable)
4	SortBufferMemTable.put(kv)	Insert into sort buffer; if full, trigger flushMemTable()
5	MergeTreeWriter.flushWriteBuffer()	Sort buffer, write to a new data file as SortedRun at level 0
6	KeyValueFileWriterFactory.create()	Create ORC/Parquet writer with proper schema and options

Step	Class.Method	Description
7	CompactManager.triggerCompaction()	Check if level-0 count exceeds threshold, submit compact task

6.3 Scenario 3: Snapshot Commit

Trigger: Checkpoint triggers prepareCommit() followed by commit()

Call Chain:

Step	Class.Method	Description
1	FileStoreWrite.prepareCommit()	Flush all in-memory data, collect new/changed file metadata
2	FileStoreCommitImpl.commit()	Begin atomic commit process
3	FileStoreCommitImpl.tryCommit()	Read latest snapshot, compute base snapshot ID
4	ManifestFile.write(entries)	Write ManifestEntry list (ADD/DELETE) to new manifest file
5	ManifestList.write(manifests)	Write manifest file list to manifest list file
6	ConflictDetection.check()	Detect partition/bucket conflicts with concurrent commits
7	Snapshot.write()	Atomically write snapshot-N file with new manifest list reference
8	CommitCallback.call()	Notify callbacks (e.g., IcebergCommitCallback for dual-write)

6.4 Scenario 4: Universal Compaction

Trigger: Level-0 sorted run count exceeds num-sorted-run.compaction-trigger

Call Chain:

Step	Class.Method	Description
1	CompactManager.triggerCompaction()	Submit async compaction task to CompactExecutor
2	UniversalCompaction.pick(levels)	Select sorted runs to compact based on size ratio
3	MergeTreeCompactTask.doCompact()	Read input sorted runs, apply merge function
4	SortMergeReader.readBatch()	Multi-way merge sort across selected sorted runs
5	MergeFunction.merge(kv1, kv2)	Apply configured merge function per key group
6	KeyValueFileWriter.write(kv)	Write merged records to new data file(s)
7	CompactResult.complete()	Return list of deleted old files and created new files

6.5 Scenario 5: Data Scan and Read

Trigger: SELECT query or Flink source function starts reading a table

Call Chain:

Step	Class.Method	Description
1	Table.newReadBuilder().newScan()	Create scan with filter/projection pushdown
2	FileStoreScan.plan()	Read manifests from latest snapshot, apply partition pruning
3	ManifestFile.read()	Deserialize ManifestEntry list from manifest files
4	ManifestEntry.filter(predicate)	Apply file-level min/max statistics for pruning
5	Plan.splits()	Group remaining DataFileMeta by partition and bucket into splits
6	TableRead.createReader(split)	Open data files, create RecordReader for a split
7	DeletionVector.isDeleted(rowId)	Apply deletion vectors to filter row-level deletes
8	MergeSortReader.read()	Multi-way merge if multiple sorted runs exist in a split

6.6 Scenario 6: Snapshot Expiration

Trigger: After commit, expire snapshots exceeding snapshot.num-retained.max

Call Chain:

Step	Class.Method	Description
1	SnapshotManager.expire()	Determine which snapshots to expire based on retention
2	SnapshotExpire.expire(snapshotId)	Process a single snapshot expiration
3	ManifestList.read()	Read manifest list of expiring snapshot
4	ManifestFile.read()	Read manifest entries to find data files to delete
5	FileStore.newDeletion().delete()	Delete orphaned data files not referenced by live snapshots
6	Snapshot.delete()	Delete the snapshot file itself

Chapter 7 Configuration and Extension Points

7.1 Core Configuration Options (CoreOptions)

CoreOptions is the central configuration class that defines all table-level options. Key configurations are organized by functional area:

Config Key	Default	Description
bucket	1	Number of buckets for hash distribution
bucket-key	(primary key)	Fields used for bucket hash computation
merge-engine	deduplicate	Merge function: deduplicate/partial-update/aggregation/first-row
write-buffer-size	256MB	Memory size for write buffer (MemTable)
page-size	64KB	Page size for columnar file formats
file.format	orc	Data file format: orc / parquet / avro
snapshot.num-retained.min	10	Minimum snapshots to retain
snapshot.num-retained.max	Integer.MAX	Maximum snapshots to retain
compaction.min.file-num	5	Min files to trigger compaction
compaction.max.file-num	50	Max files per compaction task
num-sorted-run.compaction-trigger	5	Level-0 sorted run count to trigger compaction
changelog-producer	none	Changelog mode: none / input / lookup / full-compaction
deletion-vectors.enabled	false	Enable deletion vectors for row-level delete
sequence.field	(none)	Field for sequence-based deduplication ordering

7.2 Extension Points (SPI)

paimon-core provides multiple extension points through interfaces and abstract classes:

Extension Point	Interface/Class	Description
Catalog	Catalog interface	Custom metadata store (e.g., Hive Metastore, AWS Glue)
Merge Function	MergeFunction abstract class	Custom merge logic for primary-key tables

Extension Point	Interface/Class	Description
Compact Strategy	CompactStrategy interface	Custom compaction file selection strategy
File Format	FileFormat abstract class	Custom columnar file format support
Commit Callback	CommitCallback interface	Post-commit actions (e.g., notification, CDC)
Deletion Vector	DeletionVector abstract class	Custom deletion vector implementations
Tag Callback	TagCallback interface	Actions after tag creation/deletion
Privilege Checker	PrivilegeChecker interface	Custom permission/authorization logic

Chapter 8 Design Highlights and Summary

8.1 Design Highlights

- **LSM Tree + MVCC Snapshot Isolation:** The combination of LSM tree for write-optimized storage with snapshot-based MVCC provides excellent streaming write throughput while maintaining consistent read semantics. Each snapshot is a complete, immutable view of the table state.
- **Pluggable Merge Functions:** The Strategy pattern for merge functions enables a single storage engine to serve diverse use cases — from simple deduplication to complex partial-update and aggregation scenarios — without any code changes to the core write/compact path.
- **Manifest-based Metadata:** Using a hierarchical Snapshot → ManifestList → ManifestFile → ManifestEntry design enables efficient incremental metadata updates. Only changed partitions need new manifest entries, keeping commit operations fast even for tables with millions of files.
- **Deletion Vectors for Row-level Deletes:** Instead of rewriting entire data files for deletes, deletion vectors track deleted row IDs in compact bitmap structures, enabling efficient row-level deletes on immutable columnar files.
- **Clean Layered Architecture:** The strict Table → FileStore → Operation → Storage layering with well-defined interfaces enables independent testing, evolution, and engine integration. The 41 abstract classes provide strong code reuse across implementations.
- **Iceberg Dual-Write Compatibility:** The IcebergCommitCallback enables writing Iceberg-format metadata alongside Paimon metadata, allowing tables to be read by both ecosystems without data duplication or migration.

8.2 Design Considerations

- **Complexity of SchemaManager (60KB, 1325 lines):** The SchemaManager handles schema creation, evolution, and validation in a single class. This concentration of logic may benefit from decomposition into separate SchemaCreator, SchemaEvolver, and SchemaValidator classes.
- **FileStoreCommitImpl size (51KB, 1189 lines):** The commit implementation handles normal commit, overwrite, conflict detection, and callback management. Extracting ConflictDetector and CommitCallbackManager could improve maintainability.
- **Large Catalog interface (1777 lines):** The Catalog interface includes both CRUD operations and metadata queries. Consider splitting into CatalogReader and CatalogWriter for cleaner separation of concerns.

8.3 Quality Assessment

Dimension	Rating	Description
Code Structure	★★★★☆	Clean 4-layer architecture with 36+ well-organized packages. Some oversized classes.
Design Patterns	★★★★★	Excellent use of Template Method, Strategy, Decorator, Factory, Builder, Observer patterns.
Extensibility	★★★★★	Rich SPI with 8+ extension points covering catalog, merge, compact, format, callback.
Testability	★★★★☆	Interface-based design enables mocking. Some large classes are harder to unit test.
API Design	★★★★★	Clear @Public/@Evolving annotations. Builder pattern for read/write APIs.
Documentation	★★★★☆☆	Good Javadoc on public APIs. Internal classes have less documentation coverage.

Summary: Apache Paimon's paimon-core module demonstrates a well-architected data lake storage engine with strong design pattern usage and clean layered abstractions. The LSM tree foundation combined with pluggable merge functions and manifest-based metadata management provides an excellent balance of write throughput, read performance, and operational simplicity. The codebase shows mature engineering practices with clear interface boundaries, making it accessible for both users extending the system and contributors enhancing the core.